Siraj at AISA-ArabicFC: Selective Argument Repair and the Limits of Post-Editing in Arabic Function Calling

Abdullah G. Alanazi[2,3] **Reem K. Alshammari**[1,2]
[1]King Abdulaziz City for Science and Technology, Riyadh, Saudi Arabia
[2]Artificial Intelligence Association, Riyadh, Saudi Arabia
[3]King Abdullah University of Science and Technology (KAUST), Saudi Arabia

Abstract

We describe Siraj's Track B submission to AISA-ArabicFC. A QLoRA-tuned ALLaM-7B is followed by a separately trained argument repairer whose edits pass a type-restricted acceptance filter, then by one deterministic year rule. On the blind test set (1,125 rows) it scores 0.8245 ArgEM and 0.9101 Track B, 10th of 17 systems; the post-editing stage is worth about one example on test against four on development, a difference smaller than we can distinguish from single-seed run-to-run variation. Every rule that error inspection suggested was adjudicated on out-of-fold training predictions, scored with the task's own published matcher, before it could touch development predictions; all five were rejected, one of them breaking 168 more rows than it fixed. Two development-time conclusions did not survive: the top three systems finished 5.5 to 5.7 ArgEM points ahead, so our error rate was not a task-level ceiling, and a later dataset revision lowers our development ArgEM from 0.850 to 0.8080 and neutralises our strongest rule. Post-editing rules fitted to gold conventions can inherit changes in those conventions.

1 Introduction

Arabic function calling requires selecting a tool and filling its arguments from an utterance in Modern Standard Arabic or a dialect. AISA-ArabicFC (Nacar et al., 2026a,c) scores arguments all-or-nothing (ArgEM) after a published deterministic matcher, so one mis-formatted field invalidates an otherwise correct call. After fine-tuning the residual errors look systematic—wrong year, extra qualifier, transliteration instead of Arabic—suggesting some may be amenable to post-editing. We report how that intuition was tested and where it failed: a three-stage pipeline and its measured effect (§3, §5); a gate that adjudicates candidate rules on out-of-fold training predictions (§4, §6); and a post-hoc analysis of what the standings and a post-evaluation dataset revision reveal about our own measurements (§7).

2 Task, Metric, and a Moving Target

Track B scores $0.30 \cdot \text{FnAcc} + 0.50 \cdot \text{ArgEM} + 0.20 \cdot \text{ThinkRate}$: tool-name accuracy over all rows, exact match of the argument dictionary over positive rows, and the fraction of rows containing the required reasoning trace. The data (Tuwaiq Academy, 2026a; Nacar et al., 2026b) comprise 10,550 training rows (10,500 positive), 545 development and 1,125 blind test rows, each offering four candidate tool schemas from a registry of 27 (20 appear as gold calls, seven only as distractors) and one of five dialect labels. A Track B system returns an Arabic `<think>` trace, then the selected tool and its arguments as JSON, or no call.

The organisers publish `normalize.py`, the matcher used for scoring; we re-implemented it locally and, on the release then current, reproduced the official scores of three submissions exactly. It was revised twice during development: one unchanged prediction file, submitted twice under different system names, returned 0.826 and later 0.850. A concurrent data change cannot be ruled out, so we do not attribute that to the matcher alone. Gold values recorded in our development-time error analyses differ from the current public release: values that were ISO dates now appear as natural Arabic dates. We cannot date that revision from public artefacts. All development figures below are recomputed against the current release; development-time figures are marked as such.

3 System

Figure 1 shows how the pipeline components interact.

Task model. We fine-tune `ALLaM-7B-Instruct-preview` (ALLaM-AI) with QLoRA (Dettmers et al., 2023) (r=32,

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1592–1596
October 28-29, 2026 ©2026 Association for Computational Linguistics

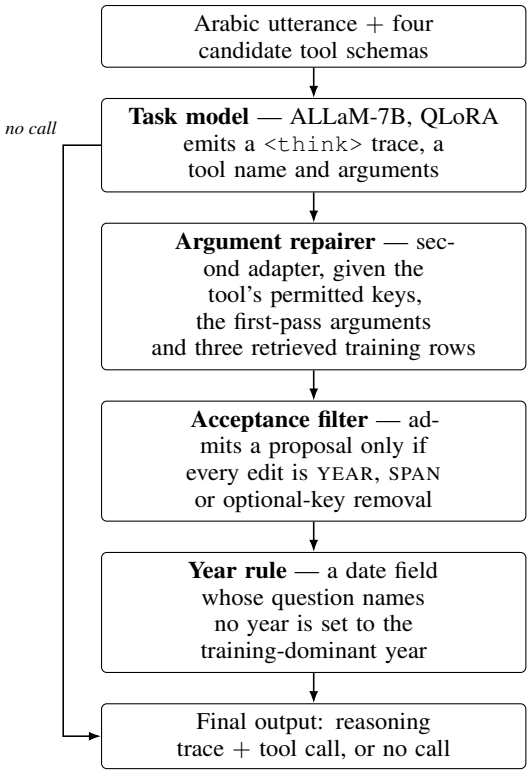


Figure 1: Siraj pipeline. The task model runs on every row; the repairer and the filter run only on rows where a tool was predicted, and the filter falls back to the first-pass arguments whenever a proposal is rejected. The year rule is applied last and is independent of the repairer.

α=64, dropout 0.05, all seven projection matrices, 4-bit NF4, bf16). The split holds only 50 no-call rows, so we hand-wrote 72 more Arabic no-call utterances over six categories—completed past actions, feelings, questions about the assistant, domain commentary, comparative questions, hypotheticals—each with a fixed rationale and grafted onto a copy of a real negative row to keep the tool inventory realistic. The 122 negatives are oversampled ninefold: 11,598 examples, 9.5% no-call. Rows without a reasoning trace receive a template sentence as the supervised target, so part of that supervision is generated. Sequences truncate at 1,280 tokens and the loss covers the full sequence. Training runs 3 epochs (2,175 steps) at learning rate 1×10^{-4}, cosine after 150 warmup steps, paged AdamW 8-bit, effective batch 16, seed 42; loss falls from 1.06 to 0.10. Checkpoint handling touches development twice: training restores the best of four checkpoints by evaluation loss on 100 shuffled development rows, and we then re-score all four with the official matcher.

Argument repairer. A second adapter (r=16, α=32, same modules) maps the question, the tool, its permitted argument keys, the first-pass arguments and three retrieved training examples to corrected arguments; retrieval is character 3-gram Jaccard over training questions of the same tool. Its loss, unlike the task model's, is masked to the target JSON; it trains 2 epochs at learning rate 7×10^{-5}, same optimiser, batch and seed. Pairs come from out-of-fold inference—the training split is halved, a model is trained on each half and predicts on the other, already-correct pairs kept as identity targets—and from corrupting training gold on a 30% sample: year, short prefix, list delimiter, field deletion, or an unused optional field.

Acceptance filter. Unfiltered, the repairer changes 47 development rows: 13 fixes, 12 breaks and 22 changes that alter no outcome. Per edit type the net effect is +3 for SPAN, +1 for REWRITE, 0 for YEAR and for optional-key removal, −1 for key addition and −2 for LANG-FLIP. The primary system admits only YEAR, SPAN and optional-key removal: each can be checked against an explicit structural relation—the same string with the year replaced, one value a substring of the other, or a key that training gold marks optional—whereas REWRITE is the residual class in which any value may replace any other, so admitting it means trusting the repairer wholesale. A proposal is further required to add only keys in the tool schema, remove only keys optional in training gold, and change at most two fields. A wide variant repairs field by field, also admitting REWRITE, and a LANG-FLIP only when it moves toward the training language policy for that field ($\geq$ 70% majority over $\geq$ 10 observations).

Deterministic rule. Date-bearing training gold carries the year 2023 in 78 of 84 cases (92.9%) while prompt dates are randomised. When a predicted date carries a year and the question mentions neither a year nor a next-year expression, we set it to 2023. This was our largest development-time gain; under the current release it is neutral (§7).

4 Validating Rules on Training Data

Development errors suggest rules; adopting them on that evidence fits a 545-row sample. Every rule error inspection suggested was therefore adjudicated on training data first: applied to the 4,000 out-of-fold predictions built for the repairer, counting

	ArgEM	Overall B	FnAcc
Development (n=545), current release			
Model alone	0.8000	0.8994	0.9982
+ repairer, wide	0.8080	0.9034	0.9982
+ repairer, primary	**0.8080**	**0.9034**	0.9982
Blind test (n=1,125), official			
Siraj (rank 10 of 17)	0.8245	0.9101	0.9929
Best system (rank 1)	0.8816	0.9400	0.9973

Table 1: ThinkRate is 1.0000 throughout. Development figures are recomputed with the current matcher and data release; test figures are the organisers' (Tuwaiq Academy, 2026b).

fixed against broken rows with the official matcher, admitted only if it fixed at least twice as many rows as it broke. Five went through this gate and all five failed (Table 2), none near the threshold—the closest still broke almost three times as many rows as it fixed. The year rule did not come from this route: it follows from a frequency in training gold, not from a hypothesis about observed errors.

5 Results

The pipeline adds +0.0080 ArgEM on development (Table 1), four of the 500 positive rows—not a fully held-out estimate, since acceptance thresholds were tuned on development and checkpoint selection touched it. On the release used during the evaluation, the model-alone and primary-pipeline files scored 0.838 and 0.850 respectively. On ArgEM alone we place 12th rather than 10th: two systems with higher ArgEM rank below us on the weighted score because of weaker function accuracy.

Variant comparison. FnAcc and ThinkRate are identical across our three submissions, so differences are entirely in ArgEM: primary filter 0.9101 Overall B, model alone 0.9097, wide filter 0.9096. Since ArgEM carries weight 0.50 and the other two components are constant, each ArgEM difference is twice the Overall B difference: the primary filter is worth about +0.0008 ArgEM over the model alone, roughly one positive test row, and the wide filter differs from it by less than one example at the reported precision.

Dialect. Development ArgEM is 0.841 MSA, 0.806 Gulf, 0.791 Levantine, 0.712 Egyptian, 0.500 Maghrebi over buckets of 323, 75, 75, 66 and 6 rows; post-editing widens the max−min gap from 0.174 to 0.341, both dominated by the six-row Maghrebi bucket where one example moves ArgEM by at least 0.167. The diagnostic track reports per-dialect *Overall* B on test, a different quantity on twice the data: 0.906 MSA, 0.922 Gulf, 0.852 Egyptian, 0.883 Levantine, 0.788 Maghrebi; macro 0.8703, 10th of 21, gap 0.134 over eleven Maghrebi rows (Tuwaiq Academy, 2026b). Gulf exceeds MSA there, so the development reading that our system favours MSA does not survive. Egyptian also trails the other nine systems with published per-dialect figures, 0.852 against 0.891–0.935. We draw no conclusion from the Maghrebi cell in either split: at six development and eleven test rows it is too small to separate a property of the system from sampling, and both max−min gaps we report are determined by it.

6 What Did Not Work

Each rule below was measured on training or out-of-fold data with the official matcher current at the time of the experiment.

Surface-form rules. Gold values are often verbatim spans of the question—city 98%, procedure and medication 93%, restaurant 92%, product 89%, category 85%, but date 56%, items 27%, currency 24%, source and target 4% and 3%—so snapping to the nearest span looks compelling, but performs poorly. So does rewriting ISO dates into the question's surface form, which would corrupt 442 rows whose gold is itself ISO, and emitting `search_type` from an explicit cue, since the gold convention omits that field far more often than the cue suggests.

Country rules. Mapping a predicted city to its country for `compare_prices` is unsafe: 21 training gold values are themselves cities. Reducing a multi-country prediction to the first mentioned country matches gold in 50 of 64 ambiguous rows, but 86 of 484 gold values are genuinely multi-country, so we did not judge the rule reliable enough to adopt.

Closed vocabularies and a learned presence gate. Snapping to closed vocabularies never triggered: only seven (tool, field) pairs have at most 25 values over at least 30 observations (eight under the current release), and the model never erred orthographically on them. A per-field TF-IDF and logistic-regression presence classifier, calibrated out-of-fold to at least 98.5% precision, left two usable gates that removed no field.

Rejected rule	fixed	broke
Verbatim-span snapping	32	200
`search_type` cue rule	8	110
Date-format harmonisation	104	442
`get_weather` days rule	9	25
Dialectal date snapping	0	2

Table 2: Rules rejected by the validation gate, on training or out-of-fold data.

7 Post-hoc Analysis

Comparison with higher-ranked systems. We concluded during development that the residual errors were annotation-convention noise and therefore irreducible. Three systems reached 0.88 ArgEM on test, 5.5 to 5.7 points above us, which shows our error rate was not a task-level ceiling—though not that they corrected the rows we missed. Our own evidence supports only the narrower claim: given this backbone and fine-tune, the remaining errors were not reducible by any deterministic rule we tested.

Sensitivity to gold conventions. Re-scoring our unchanged submissions against the current release drops development ArgEM from 0.850 to 0.8080, and the year rule—our largest development-time gain—becomes neutral (0 fixed, 0 broken, 6 rows changed): the revision replaced ISO gold dates with natural Arabic ones while our predictions stayed ISO. A rule estimated from gold conventions is only as stable as those conventions; validating on training data reduces the risk of over-fitting to a development sample, but not the risk of a change in the annotation standard.

Residual error analysis. The primary system leaves 96 erroneous development rows. Twenty-five (26%) carry a `check_in` or `check_out` mismatch, contributing 50 field-level mismatches; in 46 of those the prediction and gold name the same day and month, the prediction an ISO date carrying a year and the gold an Arabic day-and-month expression without one (row 17: `2023-02-10` against a gold reading "the tenth of February"). None names a different date, and four resisted automatic parsing. The next largest field counts run from 8 (`procedure`, `items`) to 4 (`product_name`, `category`, `date`), covering paraphrases, list separators, name transliterations, bare versus qualified currency words and definite-article presence. The residue therefore contains one sizeable date-representation mismatch plus a long tail of convention variance—not, as we had concluded, irreducible annotation noise throughout.

Test-set effect size. The repairer and filter are worth $+0.0080$ ArgEM on development (four of 500 positive rows) and about $+0.0008$ on test (one row). The sign is the same on both, but a one-row difference over a single run is not evidence of reliable transfer. Ranks 8–10 are separated by 0.0008 Overall B, so these neighbouring ranks do not support strong claims about relative method quality.

8 Conclusion

An Arabic-specialised 7B backbone with QLoRA, an out-of-fold-trained argument repairer, a precision-oriented acceptance filter and one deterministic rule reach 0.8245 ArgEM on the blind test set: 10th of 17 Track B systems on the weighted score, 12th on ArgEM alone. The restrictive filter beat the bare model there by about one example, against four on development—too small, from a single run, to call reliable transfer. Adjudicating candidate rules on training data with the official matcher kept five plausible rules out of the final system, one of which would have broken 168 more rows than it fixed. The caution is twofold: that discipline bounds only the post-editing layer and is no substitute for investing in the generator, and rules fitted to gold conventions can inherit changes in those conventions. That suggests a scope condition worth testing: a repairer of this kind should be most useful where annotation conventions are settled, and least useful on a benchmark still under revision.

Limitations

Each reported configuration was trained once at seed 42; we did not repeat training across seeds, so effects of the size we report—one to four rows—cannot be separated from run-to-run variation, and the 545-row development split and $4{,}000$ out-of-fold predictions may fail to detect a small true effect. Our conclusions are conditional on this system configuration: systems 5.5 to 5.7 ArgEM points ahead show that our error rate was not a task-level ceiling. Two components depend on development data—the per-edit-type acceptance thresholds, mitigated but not eliminated by operating over six coarse edit classes, and best-checkpoint selection on evaluation loss over 100 development rows.

Development figures are recomputed against the current public release, which differs from the one used during the evaluation; test figures are the organisers' and cannot be independently recomputed, since test gold is not released. Per-dialect gaps are unstable for the Maghrebi bucket: six development rows, eleven on test. Finally, the system was assessed only as a scoring task: an argument error costing one point here—a wrong recipient or amount—would carry real consequences in a live tool.

Ethics and Data Statement

Gradient updates used the provided training split together with the 72 no-call utterances we wrote ourselves (§3); no external corpora were used and no gold labels retrieved from external sources. We did not independently audit the organiser-provided corpus for personal or offensive content. The base model and the data were used under the terms their providers publish; we have re-distributed neither, and any adapters we release will carry terms compatible with the base model's licence. Development data influenced model and hyper-parameter selection but never gradient updates. We make available to the organisers the two LoRA adapters, an audit script that recomputes every development figure here, and a reproduction notebook pinned to `torch` 2.2.2, `transformers` 4.44.2, `peft` 0.12.0 and `bitsandbytes` 0.43.1. A clean-directory rerun on the same GPU model (NVIDIA A100-SXM4-80GB; roughly 20 GPU-hours for the project) regenerated all three submitted files bit-identically, with matching SHA-256 digests over 1,125 rows each.

Acknowledgements

We thank Tuwaiq Academy and the AISA-ArabicFC organizers for providing the dataset, the baseline model, and the evaluation framework for this shared task.

The authors used an AI assistant to draft and revise this text from their own experiment logs, run outputs and configuration files, and to write parts of the analysis code. All experiments, results, methodological choices and claims are the authors' own.

References

ALLaM-AI. ALLaM-7B-Instruct-preview. Hugging Face model.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*.

Omer Nacar, Mohammed Al Khalifa, and Saeed Alzaharani. 2026a. AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026b. From Language to Action in Arabic: Reliable Structured Tool Calling via Data-Centric Fine-Tuning. *arXiv preprint arXiv:2603.16901*.

Omer Nacar, A. Deema, and A. Mohammed. 2026c. AISA: A Unified Architecture for Agentic AI Systems.

Tuwaiq Academy. 2026a. AISA-ArabicFC dataset. Hugging Face dataset.

Tuwaiq Academy. 2026b. AISA-ArabicFC shared task leaderboard. Hugging Face Space.